



SQL — Le Cours Essentiel

Par Robert DIASSÉ



Par Robert DIASSÉ

Du debutant a l'avance — Apprendre SQL par la pratique

Fil conducteur : Tous les exemples de ce cours sont bases sur la base de donnees d'un hopital. Vous manipulez des patients, des medecins, des chambres et des consultations tout au long du cours. Cela vous permettra de suivre une logique coherente d'un chapitre a l'autre.

Avant de commencer — Qu'est-ce que SQL ?

SQL (Structured Query Language) est le langage standard pour communiquer avec une base de donnees relationnelle.

Il permet de creer des tables, d'insérer des donnees, de les lire, de les modifier et de les supprimer.

Un SGBD (Système de Gestion de Bases de Donnees) est le logiciel qui stocke et gere vos donnees. Les plus utilises sont : **MySQL, PostgreSQL, MariaDB, SQLite, Oracle, SQL Server.**

SQL se decompose en quatre grandes categories :

Categorie	Nom complet	Role	Commandes principales
LDD	Langage de Definition de Donnees	Creer et structurer les tables	CREATE, ALTER, DROP
LMD	Langage de Manipulation de Donnees	Gerer les donnees dans les tables	INSERT, SELECT, UPDATE, DELETE
LCD	Langage de Controle des Donnees	Gerer les droits d'accès	GRANT, REVOKE
LCT	Langage de Controle des Transactions	Valider ou annuler des operations	COMMIT, ROLLBACK, BEGIN

La base de donnees utilisee dans ce cours

Avant de commencer, voici le schema des tables que nous allons utiliser. Prenez le temps de le lire.

```
PATIENT      (id_pat, nom_pat, prenom_pat, date_naissance, ville, telephone)
MEDECIN      (id_med, nom_med, prenom_med, specialite, telephone_med)
SERVICE     (id_service, nom_service, batiment)
CHAMBRE      (num_chambre, type_chambre, tarif_journalier, id_service)
HOSPITALISATION (id_pat, num_chambre, date_entree, date_sortie, motif)
CONSULTATION (id_consult, date_consult, diagnostic, id_pat, id_med)
MEDICAMENT   (id_med_mt, nom_medicament, forme, prix_unitaire)
PRESCRIPTION (id_consult, id_med_mt, quantite, posologie)
```

NIVEAU DEBUTANT

1. Creer une table — CREATE TABLE

Utilite

CREATE TABLE permet de creer la structure d'une table en base de donnees.
C'est la premiere chose a faire avant de pouvoir stocker des donnees.

Syntaxe

```
CREATE TABLE nom_table (
    colonne1  TYPE  CONTRAINTE,
    colonne2  TYPE  CONTRAINTE,
    ...
);
```

Les types de donnees essentiels

Type	Description	Exemple de valeur
INT	Nombre entier	1, 42, 1000
DECIMAL(p, s)	Nombre decimal avec p chiffres dont s apres la virgule	15000.50
VARCHAR(n)	Texte de longueur variable (max n caracteres)	'Diallo', 'Dakar'
CHAR(n)	Texte de longueur fixe	'M', 'F'
DATE	Date au format AAAA-MM-JJ	'2024-03-15'
DATETIME	Date et heure	'2024-03-15 14:30:00'
TEXT	Texte long sans limite fixe	Notes medicales...
BOOLEAN	Vrai ou faux	TRUE, FALSE

Les contraintes essentielles

Contrainte	Role
PRIMARY KEY	Identifie chaque ligne de facon unique. Obligatoire sur chaque table.
NOT NULL	La colonne doit toujours avoir une valeur. Elle ne peut pas etre vide.
UNIQUE	Deux lignes ne peuvent pas avoir la meme valeur dans cette colonne.
DEFAULT valeur	Si aucune valeur n'est fournie, la valeur par default est utilisee.
CHECK (cond)	La valeur doit respecter une condition logique.
FOREIGN KEY	Lie cette colonne a la cle primaire d'une autre table.

Exemple

```
-- Creation de la table Patient
CREATE TABLE Patient (
  id_pat      INT      PRIMARY KEY,
  nom_pat     VARCHAR(50) NOT NULL,
  prenom_pat  VARCHAR(50) NOT NULL,
  date_naissance DATE,
  ville       VARCHAR(30) DEFAULT 'Dakar',
  telephone   VARCHAR(15)
);

-- Creation de la table Medecin
CREATE TABLE Medecin (
  id_med      INT      PRIMARY KEY,
  nom_med     VARCHAR(50) NOT NULL,
  prenom_med  VARCHAR(50) NOT NULL,
  specialite  VARCHAR(50),
  telephone_med VARCHAR(15)
);

-- Creation de la table Chambre avec une cle etrangere
CREATE TABLE Chambre (
  num_chambre INT      PRIMARY KEY,
  type_chambre VARCHAR(30) NOT NULL,
  tarif_journalier DECIMAL(10,2) NOT NULL,
  id_service  INT,
  FOREIGN KEY (id_service) REFERENCES Service(id_service)
);
```

A retenir : La table referencee par une **FOREIGN KEY** doit exister avant celle qui la reference.
Creez d'abord **Service** avant de creer **Chambre**.

2. Modifier et supprimer une table — ALTER TABLE, DROP TABLE

ALTER TABLE — Modifier la structure

```
-- Ajouter une colonne
ALTER TABLE Patient ADD email VARCHAR(100);

-- Modifier le type d'une colonne
ALTER TABLE Patient MODIFY telephone VARCHAR(20);

-- Supprimer une colonne
ALTER TABLE Patient DROP COLUMN email;

-- Renommer une colonne (syntaxe MySQL)
ALTER TABLE Patient RENAME COLUMN nom_pat TO nom;
```

DROP TABLE — Supprimer une table

```
-- Supprime toute la table et toutes ses donnees (irreversible !)
DROP TABLE Patient;

-- Version securisee : ne fait rien si la table n'existe pas
DROP TABLE IF EXISTS Patient;
```

Attention : `DROP TABLE` supprime definitivement la table et toutes ses donnees. Il n'y a pas de confirmation demandee. A utiliser avec beaucoup de precaution.

3. Insérer des données — INSERT INTO

Utilite

`INSERT INTO` permet d'ajouter de nouvelles lignes dans une table.

Syntaxe

```
-- Forme complete (recommandee) : on nomme les colonnes
INSERT INTO nom_table (colonne1, colonne2, ...)
VALUES (valeur1, valeur2, ...);

-- Insérer plusieurs lignes en une seule instruction
INSERT INTO nom_table (colonne1, colonne2, ...)
VALUES
    (valeur1a, valeur2a),
    (valeur1b, valeur2b),
    (valeur1c, valeur2c);
```

Exemple

```
-- Insérer un seul patient
INSERT INTO Patient (id_pat, nom_pat, prenom_pat, date_naissance, ville,
telephone)
VALUES (1, 'Diallo', 'Mamadou', '1990-05-14', 'Dakar', '77 123 45 67');

-- Insérer plusieurs medecins d'un coup
INSERT INTO Medecin (id_med, nom_med, prenom_med, specialite)
VALUES
  (1, 'Ndiaye', 'Fatou', 'Cardiologie'),
  (2, 'Sarr', 'Ibrahima', 'Pediatrie'),
  (3, 'Fall', 'Aminata', 'Gynecologie');

-- Insérer une chambre (la ville par défaut est appliquée automatiquement)
INSERT INTO Patient (id_pat, nom_pat, prenom_pat)
VALUES (2, 'Faye', 'Awa');
-- La colonne 'ville' prendra automatiquement la valeur 'Dakar' (DEFAULT)
```

Bon reflexe : Toujours nommer les colonnes dans un `INSERT INTO`.

Cela évite les erreurs si la structure de la table change un jour.

4. Lire des données — SELECT

Utilité

`SELECT` est la commande la plus utilisée en SQL. Elle permet d'interroger la base de données et d'afficher des résultats.

Syntaxe

```
SELECT colonne1, colonne2, ...
FROM nom_table;

-- Pour afficher toutes les colonnes
SELECT * FROM nom_table;
```

Exemple

```
-- Afficher tous les patients
SELECT * FROM Patient;

-- Afficher uniquement le nom et la ville des patients
SELECT nom_pat, prenom_pat, ville FROM Patient;

-- Renommer une colonne dans le résultat avec un alias
SELECT nom_pat AS Nom, prenom_pat AS Prenom FROM Patient;
```

Eliminer les doublons avec DISTINCT

```
-- Afficher la liste des villes sans repetition
SELECT DISTINCT ville FROM Patient;

-- Afficher les specialites de medecins existantes
SELECT DISTINCT specialite FROM Medecin;
```

A retenir : `SELECT *` est pratique pour decouvrir le contenu d'une table, mais dans une vraie application, il vaut mieux toujours nommer les colonnes souhaitees.

5. Filtrer les resultats — WHERE

Utilite

`WHERE` permet de ne selectionner que les lignes qui correspondent a une condition precise. Sans `WHERE`, toutes les lignes de la table sont retournees.

Syntaxe

```
SELECT colonne1, colonne2
FROM nom_table
WHERE condition;
```

Les operateurs de comparaison

Operateur	Signification	Exemple
=	Egal a	<code>ville = 'Dakar'</code>
!= ou <>	Different de	<code>ville != 'Thies'</code>
>	Superieur strictement	<code>tarif_journalier > 10000</code>
>=	Superieur ou egal	<code>tarif_journalier >= 10000</code>
<	Inferieur strictement	<code>prix_unitaire < 5000</code>
<=	Inferieur ou egal	<code>prix_unitaire <= 5000</code>

Les operateurs logiques

Operateur	Role
AND	Les deux conditions doivent etre vraies simultanement
OR	Au moins une des deux conditions doit etre vraie
NOT	Inverse la condition

Exemples

```
-- Patients de Dakar
SELECT nom_pat, prenom_pat, ville
FROM Patient
WHERE ville = 'Dakar';

-- Chambres dont le tarif est superieur a 15 000 FCFA
SELECT num_chambre, type_chambre, tarif_journalier
FROM Chambre
WHERE tarif_journalier > 15000;

-- Medecins cardiologues de la ville de Dakar (AND)
SELECT nom_med, prenom_med, specialite
FROM Medecin
WHERE specialite = 'Cardiologie' AND ville = 'Dakar';

-- Patients de Dakar ou de Thies (OR)
SELECT nom_pat, ville
FROM Patient
WHERE ville = 'Dakar' OR ville = 'Thies';

-- Patients qui ne sont pas de Dakar (NOT)
SELECT nom_pat, ville
FROM Patient
WHERE NOT ville = 'Dakar';
```

Operateurs supplementaires utiles

```
-- BETWEEN : valeur dans un intervalle (bornes incluses)
SELECT num_chambre, tarif_journalier
FROM Chambre
WHERE tarif_journalier BETWEEN 10000 AND 20000;

-- IN : la valeur appartient a une liste
SELECT nom_med, specialite
FROM Medecin
WHERE specialite IN ('Cardiologie', 'Pediatrie', 'Neurologie');

-- NOT IN : la valeur n'appartient pas a la liste
SELECT nom_med, specialite
FROM Medecin
WHERE specialite NOT IN ('Cardiologie', 'Pediatrie');

-- IS NULL : la valeur est vide (non renseignee)
SELECT nom_pat
FROM Patient
WHERE telephone IS NULL;

-- IS NOT NULL : la valeur est renseignee
```

```
SELECT nom_pat
FROM Patient
WHERE telephone IS NOT NULL;
```

6. Trier les resultats — ORDER BY

Utilite

ORDER BY permet de trier les resultats par ordre croissant (**ASC**) ou decroissant (**DESC**).

Par default, le tri est croissant.

Syntaxe

```
SELECT colonne1, colonne2
FROM nom_table
ORDER BY colonne1 ASC;    -- ou DESC
```

Exemples

```
-- Patients tries par nom dans l'ordre alphabetique
SELECT nom_pat, prenom_pat
FROM Patient
ORDER BY nom_pat ASC;

-- Chambres trieées du tarif le plus cher au moins cher
SELECT num_chambre, type_chambre, tarif_journalier
FROM Chambre
ORDER BY tarif_journalier DESC;

-- Tri sur plusieurs colonnes : d'abord par specialite, puis par nom
SELECT nom_med, prenom_med, specialite
FROM Medecin
ORDER BY specialite ASC, nom_med ASC;
```

7. Limiter les resultats — LIMIT

Utilite

LIMIT permet de n'afficher qu'un certain nombre de lignes.

Tres utile pour ne pas afficher des milliers de resultats d'un coup.

Syntaxe

```
SELECT colonnes
FROM table
LIMIT nombre;

-- Avec un decalage : sauter les n premieres lignes
SELECT colonnes
FROM table
LIMIT nombre OFFSET decalage;
```

Exemples

```
-- Afficher uniquement les 5 premiers patients
SELECT nom_pat, prenom_pat FROM Patient LIMIT 5;

-- Afficher les 3 chambres les moins cheres
SELECT num_chambre, tarif_journalier
FROM Chambre
ORDER BY tarif_journalier ASC
LIMIT 3;

-- Afficher les resultats de la page 2 (lignes 6 a 10)
SELECT nom_pat FROM Patient LIMIT 5 OFFSET 5;
```

8. Modifier et supprimer des donnees — UPDATE et DELETE

UPDATE — Modifier des donnees existantes

```
-- Syntaxe
UPDATE nom_table
SET colonne1 = nouvelle_valeur1, colonne2 = nouvelle_valeur2
WHERE condition;
```

```
-- Changer la ville d'un patient specifique
UPDATE Patient
SET ville = 'Saint-Louis'
WHERE id_pat = 1;

-- Appliquer une augmentation de 10% sur tous les tarifs des chambres VIP
UPDATE Chambre
SET tarif_journalier = tarif_journalier * 1.10
WHERE type_chambre = 'VIP';

-- Modifier plusieurs colonnes en meme temps
UPDATE Medecin
```

```
SET specialite = 'Chirurgie Generale', telephone_med = '77 999 00 11'  
WHERE id_med = 3;
```

Attention critique : Ne jamais oublier le **WHERE** dans un **UPDATE**.

Sans **WHERE**, toutes les lignes de la table seront modifiées.

DELETE — Supprimer des lignes

```
-- Syntaxe  
DELETE FROM nom_table  
WHERE condition;
```

```
-- Supprimer un patient dont l'identifiant est 5  
DELETE FROM Patient  
WHERE id_pat = 5;  
  
-- Supprimer toutes les consultations d'un medecin  
DELETE FROM Consultation  
WHERE id_med = 2;  
  
-- Vider completement une table tout en la conservant  
DELETE FROM Patient; -- Supprime toutes les lignes mais la table reste
```

Attention critique : Comme pour **UPDATE**, le **WHERE** est indispensable.

DELETE FROM Patient; supprime tous les patients sans demander confirmation.

Exercices Niveau Debutant

Exercice 1 - LDD

Creez les tables **Service** et **Hospitalisation** en respectant les types et contraintes adequats.

Service contient : id_service, nom_service, batiment.

Hospitalisation contient : id_pat, num_chambre, date_entree, date_sortie, motif.

Exercice 2 - INSERT

Inserez trois patients, deux services et cinq chambres de votre choix dans les tables correspondantes.

Exercice 3 - SELECT / WHERE

1. Afficher le nom et prenom de tous les medecins.
2. Afficher toutes les chambres dont le tarif est compris entre 8 000 et 20 000 FCFA.
3. Afficher les patients dont la ville est 'Dakar' ou 'Thies'.
4. Afficher les medicaments dont le prix est inferieur ou egal a 3 000 FCFA, tries par prix croissant.

Exercice 4 - UPDATE / DELETE

1. Le patient numero 3 a demenage a Ziguinchor. Mettez a jour sa ville.

2. Supprimez toutes les chambres de type 'Standard' dont le tarif depasse 25 000 FCFA.

NIVEAU INTERMEDIAIRE

9. Recherche textuelle — LIKE

Utilite

LIKE permet de rechercher des donnees textuelles en utilisant des motifs (patterns).

Il est utilise quand on ne connait pas l'orthographe exacte ou qu'on cherche une correspondance partielle.

Les caracteres jokers

Caractere	Role	Exemple
%	Remplace zero, un ou plusieurs caracteres	'Dia%' trouve Diallo, Diatta, Diamanka
_	Remplace exactement un seul caractere	'D_all_' trouve Diallo

Syntaxe

```
SELECT colonnes
FROM table
WHERE colonne LIKE 'motif';
```

Exemples

```
-- Patients dont le nom commence par 'D'
SELECT nom_pat, prenom_pat
FROM Patient
WHERE nom_pat LIKE 'D%';

-- Patients dont le nom se termine par 'e'
SELECT nom_pat FROM Patient
WHERE nom_pat LIKE '%e';

-- Diagnostics contenant le mot 'Infection' n'importe ou
SELECT id_consult, diagnostic
FROM Consultation
WHERE diagnostic LIKE '%Infection%';

-- Medicaments dont le nom contient exactement 5 lettres (5 underscores)
SELECT nom_medicament FROM Medicament
WHERE nom_medicament LIKE '_____';

-- NOT LIKE : exclure les patients dont le nom commence par 'F'
```

```
SELECT nom_pat FROM Patient
WHERE nom_pat NOT LIKE 'F%';
```

10. Fonctions d'agregation — COUNT, SUM, AVG, MIN, MAX

Utilite

Les fonctions d'agregation calculent une valeur statistique a partir d'un ensemble de lignes.

Fonction	Role	Exemple de resultat
COUNT(*)	Compte le nombre total de lignes	150 patients
COUNT(col)	Compte les valeurs non nulles	120 patients avec telephone
SUM(col)	Calcule la somme de toutes les valeurs	Total des tarifs
AVG(col)	Calcule la moyenne	Tarif moyen des chambres
MIN(col)	Trouve la valeur la plus petite	Chambre la moins chere
MAX(col)	Trouve la valeur la plus grande	Chambre la plus chere

Exemples

```
-- Nombre total de patients enregistres
SELECT COUNT(*) AS nombre_patients
FROM Patient;

-- Nombre de medecins ayant une specialite renseignee
SELECT COUNT(specialite) AS medecins_specialises
FROM Medecin;

-- Tarif moyen des chambres
SELECT AVG(tarif_journalier) AS tarif_moyen
FROM Chambre;

-- Chambre la moins chere et la plus chere
SELECT MIN(tarif_journalier) AS moins_cher,
       MAX(tarif_journalier) AS plus_cher
FROM Chambre;

-- Valeur totale du stock de medicaments
SELECT SUM(prix_unitaire) AS valeur_totale
FROM Medicament;
```

11. Regrouper les donnees — GROUP BY et HAVING

Utilite

GROUP BY regroupe les lignes ayant la meme valeur dans une colonne, puis applique une fonction d'agregation a chaque groupe.

HAVING filtre les resultats apres regroupement (comme un **WHERE** mais pour les groupes).

Difference entre WHERE et HAVING

Critere	WHERE	HAVING
S'applique a	Les lignes individuelles	Les groupes apres agregation
Moment d'execution	Avant le GROUP BY	Apres le GROUP BY
Peut utiliser	Colonnes normales	Fonctions d'agregation (COUNT, SUM...)

Syntaxe

```
SELECT colonne_groupe, FONCTION(colonne)
FROM table
WHERE condition_sur_lignes
GROUP BY colonne_groupe
HAVING condition_sur_groupe
ORDER BY ...;
```

Exemples

```
-- Nombre de patients par ville
SELECT ville, COUNT(*) AS nombre_patients
FROM Patient
GROUP BY ville;

-- Nombre de medecins par specialite
SELECT specialite, COUNT(*) AS effectif
FROM Medecin
GROUP BY specialite;

-- Tarif moyen des chambres par type
SELECT type_chambre, AVG(tarif_journalier) AS tarif_moyen
FROM Chambre
GROUP BY type_chambre;

-- Specialites ayant plus de 3 medecins (HAVING)
SELECT specialite, COUNT(*) AS effectif
FROM Medecin
GROUP BY specialite
HAVING COUNT(*) > 3;

-- Services avec un tarif moyen superieur a 12 000 FCFA
-- WHERE filtre avant GROUP BY, HAVING filtre apres
SELECT type_chambre, AVG(tarif_journalier) AS tarif_moyen
```

```
FROM Chambre
WHERE tarif_journalier > 0
GROUP BY type_chambre
HAVING AVG(tarif_journalier) > 12000
ORDER BY tarif_moyen DESC;
```

12. Combiner des tables — INNER JOIN

Utilite

Une base relationnelle repartit les donnees dans plusieurs tables.

JOIN permet de les reunir dans une seule requete en les liant par leurs cles.

Principe

Deux tables se joignent sur la condition : **cle_etrangere** = **cle_primaire**.

```
Patient — id_pat —> Hospitalisation
Chambre — num_chambre —> Hospitalisation
```

Syntaxe

```
SELECT table1.colonne, table2.colonne
FROM table1
INNER JOIN table2 ON table1.cle = table2.cle;

-- Avec des alias de tables (recommande pour raccourcir)
SELECT p.nom_pat, h.date_entree
FROM Patient p
INNER JOIN Hospitalisation h ON p.id_pat = h.id_pat;
```

Exemples

```
-- Nom du patient et date d'entree pour chaque hospitalisation
SELECT p.nom_pat, p.prenom_pat, h.date_entree, h.motif
FROM Patient p
INNER JOIN Hospitalisation h ON p.id_pat = h.id_pat;

-- Nom du patient + chambre + service pour chaque hospitalisation
SELECT p.nom_pat, c.type_chambre, s.nom_service
FROM Patient p
INNER JOIN Hospitalisation h ON p.id_pat = h.id_pat
INNER JOIN Chambre c ON h.num_chambre = c.num_chambre
INNER JOIN Service s ON c.id_service = s.id_service;
```

```
-- Consultations avec le nom du patient et le nom du medecin
SELECT p.nom_pat, m.nom_med, m.specialite, co.diagnostic
FROM Consultation co
INNER JOIN Patient p ON co.id_pat = p.id_pat
INNER JOIN Medecin m ON co.id_med = m.id_med;
```

A retenir : **INNER JOIN** ne retourne que les lignes qui ont une correspondance dans les deux tables.
Si un patient n'a jamais ete hospitalise, il n'apparaît pas dans le resultat.

13. Garder les lignes sans correspondance — LEFT JOIN et RIGHT JOIN

Utilite

Contrairement a **INNER JOIN**, **LEFT JOIN** retourne toutes les lignes de la table de gauche, meme si elles n'ont pas de correspondance dans la table de droite.

Les colonnes sans correspondance affichent **NULL**.

Comparaison visuelle

INNER JOIN	: retourne $A \cap B$ (intersection)
LEFT JOIN	: retourne tout A + ce qui correspond dans B
RIGHT JOIN	: retourne tout B + ce qui correspond dans A

Exemples

```
-- Tous les patients avec leurs hospitalisations, meme ceux jamais hospitalises
SELECT p.nom_pat, p.prenom_pat, h.date_entree, h.motif
FROM Patient p
LEFT JOIN Hospitalisation h ON p.id_pat = h.id_pat;
-- Les patients jamais hospitalises auront NULL dans date_entree et motif

-- Trouver les patients qui n'ont JAMAIS ete hospitalises
SELECT p.nom_pat, p.prenom_pat
FROM Patient p
LEFT JOIN Hospitalisation h ON p.id_pat = h.id_pat
WHERE h.id_pat IS NULL;

-- Tous les medecins avec leurs consultations (meme ceux n'ayant pas encore
consulte)
SELECT m.nom_med, m.specialite, COUNT(c.id_consult) AS nb_consultations
FROM Medecin m
LEFT JOIN Consultation c ON m.id_med = c.id_med
GROUP BY m.id_med, m.nom_med, m.specialite
ORDER BY nb_consultations DESC;
```

14. Recherches dans des sous-resultats — Sous-requetes

Utilite

Une sous-requete (ou requete imbriquee) est une requete **SELECT** placee a l'interieur d'une autre requete. Elle permet d'utiliser le resultat d'une premiere recherche comme critere pour une seconde.

Syntaxe

```
SELECT colonnes
FROM table
WHERE colonne IN (
    SELECT colonne
    FROM autre_table
    WHERE condition
);
```

Exemples

```
-- Patients hospitalises dans des chambres VIP
SELECT nom_pat, prenom_pat
FROM Patient
WHERE id_pat IN (
    SELECT id_pat
    FROM Hospitalisation
    WHERE num_chambre IN (
        SELECT num_chambre FROM Chambre WHERE type_chambre = 'VIP'
    )
);

-- Medecins ayant realise plus de consultations que la moyenne
SELECT nom_med, prenom_med
FROM Medecin
WHERE id_med IN (
    SELECT id_med
    FROM Consultation
    GROUP BY id_med
    HAVING COUNT(*) > (
        SELECT AVG(nb) FROM (
            SELECT COUNT(*) AS nb FROM Consultation GROUP BY id_med
        ) AS sous
    )
);

-- Chambres dont le tarif est superieur au tarif moyen
SELECT num_chambre, type_chambre, tarif_journalier
FROM Chambre
WHERE tarif_journalier > (
```

```
SELECT AVG(tarif_journalier) FROM Chambre
);
```

Conseil : Construisez d'abord la sous-requete seule pour verifier son resultat, puis integrez-la dans la requete principale.

Exercices Niveau Intermediaire

Exercice 5 - LIKE

1. Afficher tous les patients dont le nom commence par 'D'.
2. Afficher les diagnostics qui contiennent le mot 'fièvre'.
3. Afficher les médicaments dont le nom se termine par 'ine'.

Exercice 6 - Agregations

1. Compter le nombre total de chambres par type.
2. Calculer le tarif journalier moyen, minimum et maximum des chambres.
3. Afficher les villes ayant plus de 10 patients enregistrés.

Exercice 7 - Jointures

1. Afficher le nom des patients et le diagnostic de chaque consultation.
2. Afficher pour chaque chambre son type et le nom du service auquel elle appartient.
3. Afficher les medecins n'ayant realise aucune consultation.

Exercice 8 - Sous-requetes

1. Afficher les patients hospitalises dans le service 'Urgences'.
2. Afficher les chambres dont le tarif est superieur au tarif moyen.
3. Afficher les medecins ayant au moins une consultation avec le diagnostic 'Paludisme'.

NIVEAU AVANCE

15. Combiner des ensembles de resultats — UNION, INTERSECT, EXCEPT

Utilite

Ces operateurs combinent les resultats de deux requetes **SELECT**.

Les deux requetes doivent retourner le meme nombre de colonnes avec des types compatibles.

Operateur	Role
UNION	Combine les resultats des deux requetes en supprimant les doublons
UNION ALL	Combine les resultats en conservant les doublons

Opérateur	Rôle
INTERSECT	Retourne uniquement les lignes presentes dans les deux resultats
EXCEPT	Retourne les lignes du premier resultat absentes du second

Exemples

```
-- Liste de tous les noms : patients ET medecins (UNION supprime les doublons)
SELECT nom_pat AS nom FROM Patient
UNION
SELECT nom_med AS nom FROM Medecin;

-- UNION ALL : garder tous les enregistrements meme les noms identiques
SELECT nom_pat AS nom, 'Patient' AS type FROM Patient
UNION ALL
SELECT nom_med AS nom, 'Medecin' AS type FROM Medecin;

-- Patients ayant ete a la fois hospitalises ET en consultation
SELECT id_pat FROM Hospitalisation
INTERSECT
SELECT id_pat FROM Consultation;

-- Patients hospitalises mais n'ayant jamais eu de consultation
SELECT id_pat FROM Hospitalisation
EXCEPT
SELECT id_pat FROM Consultation;
```

16. Creer des vues — CREATE VIEW

Utilite

Une vue est une requete enregistree sous un nom, qu'on peut ensuite utiliser comme une table. Elle ne stocke pas les donnees, elle les affiche dynamiquement a chaque appel.

Avantages : Simplifier les requetes complexes, masquer les details de structure, securiser l'accès.

Syntaxe

```
-- Creer une vue
CREATE VIEW nom_vue AS
SELECT ...;

-- Utiliser une vue comme une table
SELECT * FROM nom_vue;

-- Supprimer une vue
DROP VIEW nom_vue;
```

Exemples

```
-- Vue : patients actuellement hospitalises avec leur chambre et service
CREATE VIEW v_patients_hospitalises AS
SELECT p.nom_pat, p.prenom_pat, c.type_chambre, s.nom_service,
       h.date_entree, h.motif
FROM Patient p
INNER JOIN Hospitalisation h ON p.id_pat      = h.id_pat
INNER JOIN Chambre         c ON h.num_chambre = c.num_chambre
INNER JOIN Service         s ON c.id_service  = s.id_service
WHERE h.date_sortie IS NULL; -- uniquement les sejours en cours

-- Utilisation : simple comme une table
SELECT * FROM v_patients_hospitalises;
SELECT * FROM v_patients_hospitalises WHERE nom_service = 'Cardiologie';

-- Vue : resume des medecins et leur nombre de consultations
CREATE VIEW v_activite_medecins AS
SELECT m.nom_med, m.prenom_med, m.specialite,
       COUNT(c.id_consult) AS total_consultations
FROM Medecin m
LEFT JOIN Consultation c ON m.id_med = c.id_med
GROUP BY m.id_med, m.nom_med, m.prenom_med, m.specialite;
```

17. Logique conditionnelle — CASE WHEN

Utilite

CASE WHEN permet d'ajouter de la logique conditionnelle directement dans une requete SQL.
C'est l'equivalent d'un **if / else if / else** en SQL.

Syntaxe

```
CASE
  WHEN condition1 THEN resultat1
  WHEN condition2 THEN resultat2
  ELSE resultat_par_defaut
END AS alias
```

Exemples

```
-- Categoriser les chambres par gamme de tarif
SELECT num_chambre, tarif_journalier,
       CASE
         WHEN tarif_journalier < 10000 THEN 'Economique'
         WHEN tarif_journalier < 20000 THEN 'Standard'
```

```
        WHEN tarif_journalier < 40000 THEN 'Confort'
        ELSE 'Prestige'
    END AS categorie
FROM Chambre
ORDER BY tarif_journalier;

-- Afficher l'etat d'une hospitalisation (en cours ou terminee)
SELECT p.nom_pat, h.date_entree,
       CASE
           WHEN h.date_sortie IS NULL THEN 'En cours'
           ELSE 'Termine'
       END AS statut_sejour
FROM Patient p
INNER JOIN Hospitalisation h ON p.id_pat = h.id_pat;

-- Compter les consultations par tranche horaire (matin / apres-midi)
SELECT
    CASE
        WHEN HOUR(date_consult) < 12 THEN 'Matin'
        ELSE 'Apres-midi'
    END AS periode,
    COUNT(*) AS nb_consultations
FROM Consultation
GROUP BY periode;
```

18. Verifier l'existence — EXISTS et NOT EXISTS

Utilite

EXISTS verifie si une sous-requete retourne au moins une ligne.

Il est souvent plus rapide que **IN** sur de grandes tables car il s'arrete des qu'il trouve une correspondance.

Exemples

```
-- Medecins ayant realise au moins une consultation
SELECT nom_med, specialite
FROM Medecin m
WHERE EXISTS (
    SELECT 1 FROM Consultation c
    WHERE c.id_med = m.id_med
);

-- Patients n'ayant jamais ete hospitalises
SELECT nom_pat, prenom_pat
FROM Patient p
WHERE NOT EXISTS (
    SELECT 1 FROM Hospitalisation h
    WHERE h.id_pat = p.id_pat
);
```

```
-- Services ayant au moins une chambre disponible
SELECT nom_service
FROM Service s
WHERE EXISTS (
    SELECT 1 FROM Chambre c
    WHERE c.id_service = s.id_service
);
```

19. Securiser les operations — Transactions

Utilite

Une transaction regroupe plusieurs operations SQL en un bloc indivisible.

Soit toutes les operations reussissent (**COMMIT**), soit elles sont toutes annulees (**ROLLBACK**).

C'est le principe **ACID** : Atomicite, Coherence, Isolation, Durabilite.

Cas concret

Transférer un patient d'une chambre à une autre nécessite deux operations :

libérer l'ancienne chambre ET réserver la nouvelle.

Si l'une échoue, aucune des deux ne doit être appliquée.

Syntaxe

```
START TRANSACTION;  -- (ou BEGIN)

-- Operations à exécuter ensemble
UPDATE ...;
INSERT ...;
DELETE ...;

COMMIT;  -- Valider définitivement toutes les operations
-- ou
ROLLBACK; -- Annuler tout si une erreur s'est produite
```

Exemple

```
-- Transfert du patient 5 de la chambre 101 vers la chambre 205
START TRANSACTION;

-- Etape 1 : Enregistrer la date de sortie de l'ancienne chambre
UPDATE Hospitalisation
SET date_sortie = CURDATE()
WHERE id_pat = 5 AND num_chambre = 101;

-- Etape 2 : Créer une nouvelle entrée d'hospitalisation
INSERT INTO Hospitalisation (id_pat, num_chambre, date_entree, motif)
```

```
VALUES (5, 205, CURDATE(), 'Transfert de service');

-- Si les deux etapes ont reussi, on valide
COMMIT;

-- En cas d'erreur dans le code applicatif, on pourrait faire :
-- ROLLBACK; pour annuler les deux operations
```

20. Accelerer les recherches — INDEX

Utilite

Un index est une structure de donnees qui accelere les recherches dans une table.

Sans index, le SGBD lit toutes les lignes une par une (full scan).

Avec un index, il trouve directement les lignes concernees (comme un index de livre).

Quand creer un index ? Sur les colonnes frequemment utilisees dans des **WHERE**, **JOIN** ou **ORDER BY**.

Syntaxe

```
-- Creer un index simple
CREATE INDEX nom_index ON nom_table (colonne);

-- Creer un index unique (valeurs uniques obligatoires)
CREATE UNIQUE INDEX nom_index ON nom_table (colonne);

-- Supprimer un index
DROP INDEX nom_index ON nom_table;

-- Voir les index existants sur une table
SHOW INDEX FROM nom_table;
```

Exemple

```
-- Index sur la ville des patients : les recherches par ville seront beaucoup plus rapides
CREATE INDEX idx_patient_ville ON Patient (ville);

-- Index sur la specialite des medecins
CREATE INDEX idx_medecin_specialite ON Medecin (specialite);

-- Index composite : accelere les recherches combinant les deux colonnes
CREATE INDEX idx_hospit_dates ON Hospitalisation (date_entree, date_sortie);
```

A savoir : Les index accelèrent les **SELECT** mais ralentissent légèrement les **INSERT**, **UPDATE** et **DELETE** car le SGBD doit mettre à jour l'index à chaque modification.

Ne creez pas d'index sur toutes les colonnes sans reflexion.

21. Requetes avancees avec sous-requetes correes

Utilite

Une sous-requete corree est une sous-requete qui reference une colonne de la requete principale. Elle est re-executee pour chaque ligne de la requete principale.

Exemples

```
-- Patients dont le nombre de consultations est superieur a la moyenne
SELECT p.nom_pat, p.prenom_pat,
       (SELECT COUNT(*) FROM Consultation c WHERE c.id_pat = p.id_pat) AS nb_consult
FROM Patient p
WHERE (
    SELECT COUNT(*) FROM Consultation c WHERE c.id_pat = p.id_pat
) > (
    SELECT AVG(cnt) FROM (
        SELECT COUNT(*) AS cnt FROM Consultation GROUP BY id_pat
    ) AS stats
);

-- Pour chaque medecin : son nombre de consultations par rapport au total
SELECT m.nom_med, m.specialite,
       COUNT(c.id_consult) AS nb_consult,
       ROUND(COUNT(c.id_consult) * 100.0 / (SELECT COUNT(*) FROM Consultation), 2) AS
pourcentage
FROM Medecin m
LEFT JOIN Consultation c ON m.id_med = c.id_med
GROUP BY m.id_med, m.nom_med, m.specialite
ORDER BY nb_consult DESC;
```

Tableau Recapitulatif — Toutes les Commandes SQL

LDD — Langage de Definition de Donnees

Commande	Role
CREATE TABLE	Creer une table
ALTER TABLE	Modifier la structure d'une table
DROP TABLE	Supprimer une table
CREATE INDEX	Creer un index
CREATE VIEW	Creer une vue

LMD — Langage de Manipulation de Donnees

Commande	Role
INSERT INTO	Inserer des donnees
SELECT	Lire / interroger des donnees
UPDATE	Modifier des donnees existantes
DELETE FROM	Supprimer des lignes

Clauses SELECT

Clause	Role
WHERE	Filtrer les lignes selon une condition
ORDER BY	Trier les resultats
LIMIT	Limiter le nombre de resultats
DISTINCT	Supprimer les doublons
GROUP BY	Regrouper les lignes par valeur
HAVING	Filtrer les groupes
INNER JOIN	Jointure sur correspondance
LEFT JOIN	Jointure en conservant toutes les lignes gauches
LIKE	Recherche par motif textuel
IN / NOT IN	Tester l'appartenance a une liste
BETWEEN	Tester un intervalle
IS NULL	Tester la valeur nulle
EXISTS	Verifier l'existence dans une sous-requete
CASE WHEN	Logique conditionnelle
UNION	Combiner deux resultats

LCT — Transactions

Commande	Role
START TRANSACTION / BEGIN	Demarrer une transaction
COMMIT	Valider definitivement
ROLLBACK	Annuler toutes les operations

Exercices Niveau Avance

Exercice 9 - CASE WHEN

1. Afficher le nom de chaque patient avec une colonne 'tranche_age' :
'Enfant' si age < 15 ans, 'Adulte' si entre 15 et 60 ans, 'Senior' sinon.
2. Categoriser les medicaments en 'Peu cher' (< 2000), 'Moyen prix' (entre 2000 et 8000) et 'Couteux' (> 8000).

Exercice 10 - Vues

1. Creer une vue `v_consultations_detail` qui affiche pour chaque consultation :
le nom du patient, le nom du medecin, sa specialite, la date et le diagnostic.
2. Creer une vue `v_occupation_service` qui affiche le nombre de chambres par service.

Exercice 11 - Transactions

Ecrire la transaction permettant d'enregistrer la sortie d'un patient de l'hopital :

1. Mettre a jour la date de sortie dans `Hospitalisation`.
 2. Insérer une consultation de sortie dans `Consultation`.
- Ces deux operations doivent etre atomiques.

Exercice 12 - Requetes avancees

1. Afficher les medecins dont toutes les consultations ont abouti au diagnostic 'Paludisme'.
2. Afficher le top 3 des medecins ayant le plus grand nombre de consultations avec leur nombre.
3. Calculer pour chaque service le taux d'occupation : (chambres occupees / total chambres) * 100.

*Ce cours couvre l'ensemble des notions SQL necessaires du debutant a l'avance,
avec des exemples concrets issus d'un systeme de gestion hospitaliere.
Les exercices proposes permettent de consolider progressivement chaque notion abordee.*

Robert DIASSE | Ingenieur Logiciel et Data | Idiasse@groupeisi.com

Groupe ISI — Institut Superieur d'Informatique | Annee academique 2025-2026